



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year:2024), B.Sc. in CSE (Day)

Lab Report No : 03

Course Title: Algorithm Lab

Course Code: CSE 206

Section: 223-D5

Lab Experiment Name :

Implement Dijkstra using an adjacency list and print the shortest path from source to destination.

Implement Dijkstra using an adjacency matrix and find how many shortest paths are there from source to destination.

Student Details

Name		ID
1.	Md. Reahoon Zannah	223002038

Lab Date : 09-10-2024

Submission Date : 14-10-2024

Course Teacher's Name : Md. Sabbir Hosen Mamun

Lab Report Status

Marks :

Signature :

Comment :

Date :

Title:

- Implement Dijkstra using an adjacency list and print the shortest path from source to destination.
- Implement Dijkstra using an adjacency matrix and find how many shortest paths are there from source to destination.

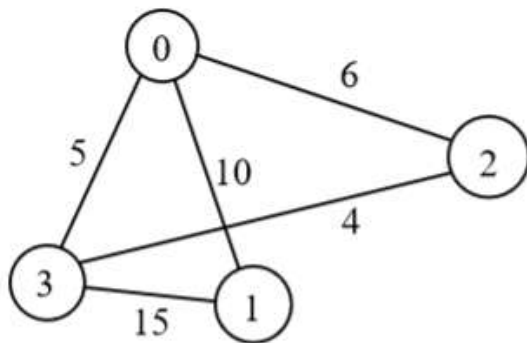
Objective:

The objective of this project is to implement Dijkstra's algorithm for finding the shortest path in weighted, directed graphs using two different representations: an adjacency list and an adjacency matrix. The project aims to demonstrate the efficiency and effectiveness of Dijkstra's algorithm in real-world applications by printing the shortest path from a given source to a destination using the adjacency list and determining the number of distinct shortest paths from the source to the destination using the adjacency matrix.

Problem 1

IMPLEMENTATION:

Implement Dijkstra using an adjacency list and print the shortest path from source to destination.



Graph

```

import java.util.*;

class Graph {

    private int vertices;

    private LinkedList<Edge>[] adjacencyList;

    static class Edge {

        int vertex;

        int weight;

        Edge(int vertex, int weight) {

            this.vertex = vertex;

            this.weight = weight;

        }

    }

    Graph(int vertices) {

        this.vertices = vertices;

        adjacencyList = new LinkedList[vertices];

        for (int i = 0; i < vertices; i++) {

            adjacencyList[i] = new LinkedList<>();

        }

    }

    void addEdge(int source, int destination, int weight) {

        adjacencyList[source].add(new Edge(destination, weight));

        adjacencyList[destination].add(new Edge(source, weight));

    }

    void dijkstra(int source, int destination) {

        PriorityQueue<Edge> priorityQueue = new PriorityQueue<>(Comparator.comparingInt(edge ->
edge.weight));

        int[] distances = new int[vertices];

```

```

int[] predecessors = new int[vertices];
Arrays.fill(distances, Integer.MAX_VALUE);
Arrays.fill(predecessors, -1);
distances[source] = 0;
priorityQueue.add(new Edge(source, 0));

while (!priorityQueue.isEmpty()) {
    Edge edge = priorityQueue.poll();
    int vertex = edge.vertex;

    for (Edge adjacentEdge : adjacencyList[vertex]) {
        int adjVertex = adjacentEdge.vertex;
        int weight = adjacentEdge.weight;
        int newDist = distances[vertex] + weight;

        if (newDist < distances[adjVertex]) {
            distances[adjVertex] = newDist;
            predecessors[adjVertex] = vertex;
            priorityQueue.add(new Edge(adjVertex, newDist));
        }
    }
}

printPath(predecessors, destination);
System.out.println("Shortest path distance: " + distances[destination]);
}

void printPath(int[] predecessors, int destination) {
    if (predecessors[destination] == -1) {
        System.out.print("No path found");
    }
}

```

```

        return;
    }

    List<Integer> path = new ArrayList<>();
    for (int at = destination; at != -1; at = predecessors[at]) {
        path.add(at);
    }

    Collections.reverse(path);

    System.out.println("Shortest path: " + path);
}

public static void main(String[] args) {
    Graph graph = new Graph(4);
    graph.addEdge(0, 1, 10);
    graph.addEdge(0, 2, 6);
    graph.addEdge(0, 3, 5);
    graph.addEdge(1, 3, 15);
    graph.addEdge(2, 3, 4);

    int source = 0; // Starting node
    int destination = 1; // Ending node
    graph.dijkstra(source, destination);
}
}

```

Output:

```
Note: /tmp/zQSueYb1G0/Graph.java uses unchecked or unsafe
operations.
Note: Recompile with -Xlint:unchecked for details.
java -cp /tmp/zQSueYb1G0/Graph
Shortest path: [0, 1]
Shortest path distance: 10

=== Code Execution Successful ===
```

Problem 2

IMPLEMENTATION:

Implement Dijkstra using an adjacency matrix and find how many shortest paths are there from source to destination.

(reference same graph)

```
import java.util.Arrays;
import java.util.PriorityQueue;
import java.util.Comparator;
class Dijkstra {
    private int vertices;
    private int[][] adjacencyMatrix;
    Dijkstra(int vertices) {
        this.vertices = vertices;
        adjacencyMatrix = new int[vertices][vertices];
        for (int i = 0; i < vertices; i++) {
            Arrays.fill(adjacencyMatrix[i], Integer.MAX_VALUE);
```

```

        adjacencyMatrix[i][i] = 0;
    }
}

void addEdge(int source, int destination, int weight) {
    adjacencyMatrix[source][destination] = weight;
    adjacencyMatrix[destination][source] = weight;
}

void dijkstra(int source, int destination) {
    int[] distances = new int[vertices];
    int[] pathCount = new int[vertices];
    Arrays.fill(distances, Integer.MAX_VALUE);
    distances[source] = 0;
    pathCount[source] = 1;

    PriorityQueue<int[]> priorityQueue = new
PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    priorityQueue.add(new int[]{source, 0});
    while (!priorityQueue.isEmpty()) {
        int[] current = priorityQueue.poll();
        int node = current[0];

        for (int i = 0; i < vertices; i++) {
            if (adjacencyMatrix[node][i] != Integer.MAX_VALUE) {
                int newDist = distances[node] + adjacencyMatrix[node][i];
                if (newDist < distances[i]) {
                    distances[i] = newDist;

```

```

        pathCount[i] = pathCount[node];
        priorityQueue.add(new int[]{i, newDist});
    } else if (newDist == distances[i]) {
        pathCount[i] += pathCount[node];
    }
}
}
}

System.out.println("Shortest path distance: " + distances[destination]);
System.out.println("Number of shortest paths: " + pathCount[destination]);
}

```

```

public static void main(String[] args) {
    Dijkstra graph = new Dijkstra(4);
    graph.addEdge(0, 1, 10);
    graph.addEdge(0, 2, 6);
    graph.addEdge(0, 3, 5);
    graph.addEdge(1, 3, 15);
    graph.addEdge(2, 3, 4);

    int source = 0; // Starting node
    int destination = 1; // Ending node
    graph.dijkstra(source, destination);
}
}

```

Output:


```
java -cp /tmp/0drlREeF9W/Dijkstra
Shortest path distance: 10
Number of shortest paths: 4

=== Code Execution Successful ===
```

Summary:

This project provides comprehensive implementations of Dijkstra's algorithm, one of the most widely used algorithms for solving the single-source shortest path problem in graphs. The first implementation utilizes an adjacency list representation, which facilitates efficient edge traversal and allows for the direct printing of the shortest path from the specified source to the destination. This implementation emphasizes the algorithm's ability to handle sparse graphs effectively, showcasing its utility in various applications such as navigation systems and network routing.

The second implementation employs an adjacency matrix representation of the graph, focusing on calculating the number of distinct shortest paths from the source to the destination. By leveraging the matrix structure, this implementation explores the connectivity of vertices and tracks the multiplicity of shortest paths, providing insights into the graph's topology. This approach enhances the understanding of the relationship between graph structure and pathfinding.